

QPROG Project

Implementation of Modular Exponentiation

Mateus de Oliveira Oliveira
Department of Computer and Systems Sciences
Stockholm University

1 Modular Exponentiation

Modular exponentiation is the problem of computing the value $X^Y \bmod N$ for given positive integers X , Y and N . Modular exponentiation is a central subroutine of Shor's period finding algorithm, which by its turn is used for integer factorization. In this project, you will be implementing a quantum circuit for modular exponentiation from scratch. The implementation should be done using qiskit and should comply with the steps listed below. Each step specifies a function that takes a quantum circuit as input (together with other parameters) and applies the gates corresponding to a suitable unitary transformation.

- Unless stated otherwise, you are only allowed to use the gates `X`, `CNOT`, `CCNOT` and `Multicontrolled-NOT`. See Appendix A for an example of use of the `Multicontrolled-NOT` gate.
- Please feel free to implement additional auxiliary functions as long as the functions listed below are implemented.
- For each implemented function, please give evidence that the implementation is correct, by initializing each input register with some number of up to 4 bits, each auxiliary register with $|0\rangle$, and by measuring the output register, to verify if the value is as expected. **The tests should be described in a PDF report. If for some reason you were not able to run the simulation of a function (for example, if the number of qubits became too large), then specify this in your report.**
- Please try to reuse qubits from auxiliary registers as much as possible. For this, it is crucial that the auxiliary registers are equal to $|0\rangle$ both at the beginning and at the end of the computation of each function.
- The value of each question is expressed as a percentage of the total points for the project.

Glossary

1. `circuit` denotes the quantum circuit to be modified by the function.
2. Lower case letters `a,b,c,...` denote integers specifying single-qubit registers. The letters `x,y,z,...` denote bits.
3. Upper case letters `A,B,C,...` denote lists of integers, which intuitively specify qubit registers used to store the input and output of functions. The letters `X,Y,Z,...` denote numbers represented in binary. `len(A)` denotes the length of the list `A`. `A[0]` is the least significant bit-index of the register, while `A[len(A)-1]` is the most significant bit-index. `bin(A)` denotes the binary string stored in the register `A`. `number(A)` denotes the number represented by `bin(A)`.
4. `AUX` is a list of integers which is intuitively used to store the partial results of the computation. Unless stated otherwise, it is assumed that `aux` is initialized with the state $|0\rangle$ before the function is applied, and that this register is reset to $|0\rangle$ once the function has been applied.

1.1 Initialization (7%)

The function `set_bits(circuit,A,X)` initializes the bits of register `A` with the binary string `X`. In other words, for each i in `len(X)`, if `X[i]=1`, then the function applies the **X**-gate to `A[i]`. Otherwise, it does nothing. Assume `len(A)=len(X)`. For instance, if `qubits = [2,4,3,7,5]` and `X = 01011`, then you will apply an **X**-gate to the qubits 4, 7 and 5.

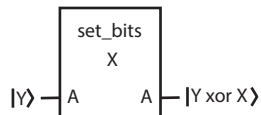


Figure 1: `set_bits`

1.2 Copy (7%)

Function `copy(circuit,A,B)` copies the binary string `bin(A)` to register `B`. Assume that `len(A)=len(B)` and that before the application of the function, `B` is initialized to $|0\rangle$. **Hint:** use CNOT gates.

1.3 Full Adder (7%)

Function `full_adder(circuit,a,b,r,c_in,c_out,AUX)` implements a full adder (Figure 3). The registers `a` and `b` store the bits to be added, `c_in` stores the carry-in bit, `c_out` stores the carry-out bit, `r` stores the result of the sum, and `AUX` is the auxiliary register.

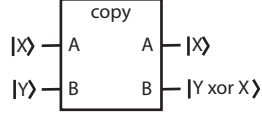


Figure 2: copy

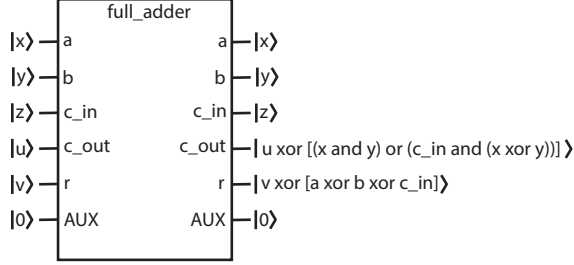


Figure 3: full_adder

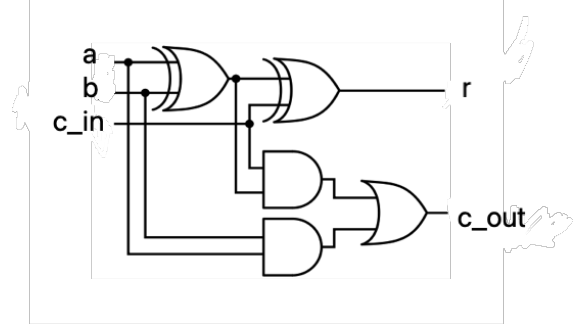


Figure 4: Full Adder Classical Circuit

1.4 Addition (7%)

Function `add(circuit,A,B,R,AUX)` implements a circuit that adds `number(A)` to `number(B)` and stores the result at register `R`. Assume that `len(A)=len(B)=len(R)`. Such a circuit is obtained by creating a cascade of full-adder circuits, as specified in Figure 5. The carry bits are part of the auxiliary register `AUX`. Note that the carry-in bit of the first adder (from right to left) is set to 0.

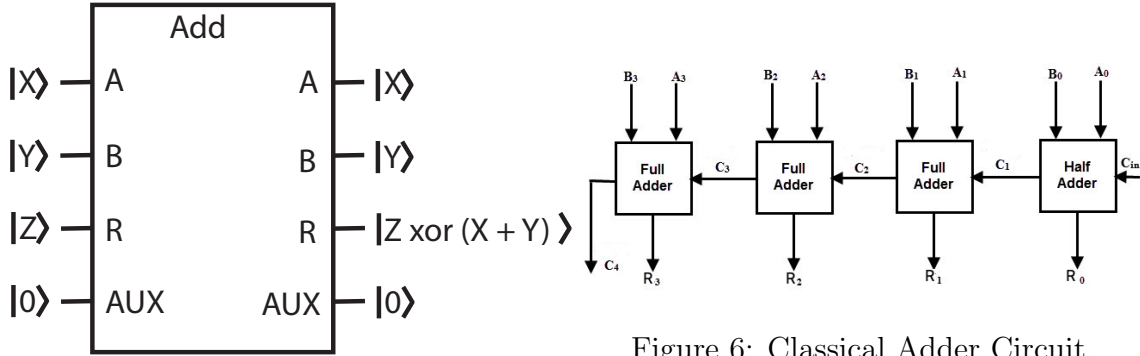


Figure 5: add

Figure 6: Classical Adder Circuit.

1.5 Subtraction (7%)

Function `subtract(circuit,A,B,R,AUX)` implements a circuit that subtracts `Number(B)` from `Number(A)` and stores the result in register `R`. Assume that `len(A) = len(B) = len(R)`.

Such a circuit can be obtained by negating each bit stored in B , and by applying the adder circuit with the first carry-in bit set to 1 instead of 0 (Figure 9).

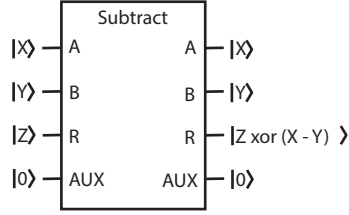


Figure 7: add

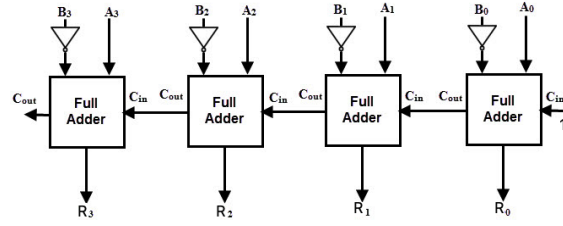


Figure 8: Classical Subtractor Circuit

1.6 Comparison (7%)

The function `greater_or_eq(circuit,A,B,r,AUX)` implements a circuit that tests whether `number(A)` is greater than or equal to `number(B)`. The result is stored in register `r`. Assume `len(A) = len(B)`.

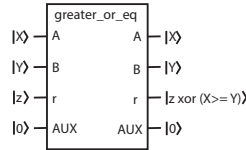


Figure 9: `greater_or_eq`

1.7 Addition Modulo N (7%)

The function `add_mod(circuit,N,A,B,R,aux)` implements a circuit that adds `number(A)` to `number(B)` modulo `number(N)`. The result is stored at register `R`.

Assume that `len(A)=len(B)=len(N)`, `number(A)<number(N)` and `number(B)<number(N)`.

Hint: Since the numbers stored in A and in B are smaller than N the sum will be smaller than $2N$. Therefore, it is enough to add both numbers, test whether the result is greater than or equal to N , and if it is, subtract N from the result. Note that the test can be done by applying a controlled subtraction, where the control bit is the output bit of the comparison function.

1.8 Multiplication by Two Modulo N (7%)

The function `times_two_mod(circuit,N,A,R,AUX)` implements a circuit that doubles `number(A)` modulo `number(N)`. The result is stored in the register `R`.

Hint: copy the register `A` and then compute `number(A)+number(A)` modulo `number(N)`.

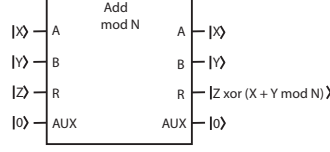


Figure 10: add_mod

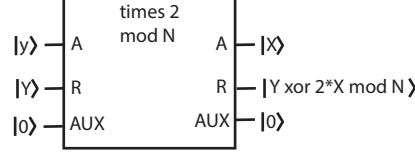


Figure 11: times_two_mod

1.9 Multiplication by a Power of Two Modulo N (7%)

The function `times_two_power_mod(circuit,N,A,k,R,AUX)` implements a circuit that multiplies `number(A)` by 2^k modulo `number(N)`.

Hint: Apply the function `double_mod(circuit,N,A,R,AUX)` k times in a row.

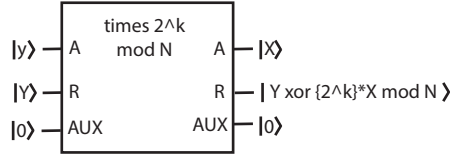


Figure 12: times_two_power_mod

1.10 Multiplication Modulo N (7%)

The function `multiply_mod(circuit,N,A,B,R,AUX)` implements a circuit that multiplies `number(A)` with `number(B)` modulo `number(N)`. The result is stored in the register R.

Hint: $\text{Number}(A) \cdot \text{Number}(B)$ is equal to $\text{Number}(A) \cdot \sum_{k=0}^{\text{len}(B)-1} 2^k \cdot \text{Number}(B)$. Therefore, apply a circuit that computes the partial sums for k varying between 0 and $\text{len}(B)-1$. More specifically, at each step k , apply the function `multiply_power_two_mod(circuit,N,A,k,R,AUX)` with control bit $B[k]$ and sum the result with the partial sum obtained at step $k-1$.

1.11 Multiplication Modulo N with a hard-coded factor (10%)

The function `multiply_mod_fixed(circuit,N,X,B,AUX)` implements a circuit that multiplies `number(B)` by a fixed number X modulo N . The caveat is that this multiplication should be done in place. That is, the circuit implements a unitary transformation that sends $|\text{number}(B)\rangle|0\rangle$ to $|X \cdot \text{number}(B) \bmod N\rangle|0\rangle$.

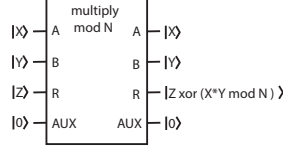


Figure 13: multiply_mod

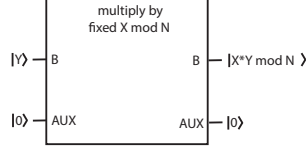


Figure 14: multiply_mod_fixed

1.12 Multiplication by $X^{2^k} \bmod N$ (10%)

Let X be a fixed n -bit number. The function

```
multiply_mod_fixed_power_2_k(circuit,N,X,B,AUX,k)
```

implements a circuit that multiplies `number(B)` by the number X^{2^k} modulo N . Important note: instead of applying the function of the previous section 2^k times, first pre-compute the number $W = X^{2^k} \bmod N$ using python (not a quantum circuit), and then call the function `multiply_mod_fixed(circuit,N,W,B,AUX)` that constructs the circuit that multiplies `number(B)` by W . Observe that the number W can be efficiently computed by taking the square of $X \bmod N$, then the square of the resulting number $\bmod N$ and so on.

1.13 Multiplication by $X^Y \bmod N$ (10%)

Let X be a fixed n -bit number. The function

```
multiply_mod_fixed_power_Y(circuit,N,X,B,AUX,Y)
```

Implements a circuit that multiplies `number(B)` by the number X^Y modulo N , where Y is a given n -bit number. Important note: if $Y = y_{n-1} \dots y_1 y_0$, then $Y = \sum_{k=0}^n y_k \cdot 2^k$. Therefore, to implement this circuit, for each $k \in \{0, \dots, n-1\}$, view the bit y_k as a control bit. If this bit is 0 the circuit does nothing. If it is 1, apply the function of Section 1.12 to multiply the number `number(B)` by X^{2^k} modulo N .

2 Bonus Challenge (10%)

Using the circuit you implemented in Section 1.12, implement the algorithm for period finding. This is a bonus question. Depending on the number of qubits used in your im-

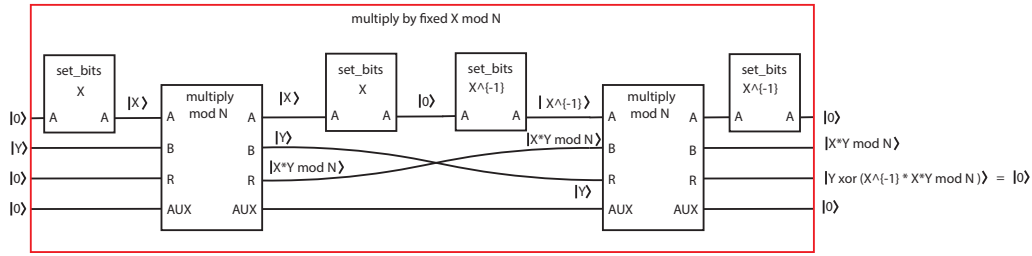


Figure 15: `multiply_mod_fixed`: Implementation details.

plementation of modular exponentiation, it is not clear that you will be able to simulate the resulting circuit neither in a local simulator nor in the cloud. But you can try it. For this question, you are allowed to use more advanced functions of qiskit, such as the native implementation of QFT.

A Multicontrolled NOT Gate

To be able to use multicontrolled NOT gates you need to import the `MCXGate` module. This gate applies the NOT gate to the target qubit if and only if all control qubits are set to $|1\rangle$. An example is provided below where the control qubits are 2, 3 and 4 and the target qubit is 7.

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import MCXGate
# Create a quantum circuit with at least 8 qubits
qc = QuantumCircuit(8)
# Define the number of control qubits
num_controls = 3
# Create an MCX gate
mcx_gate = MCXGate(num_controls)
# Apply the MCX gate (control qubits 2, 3, 4 and target qubit 7)
qc.append(mcx_gate, qargs=[2, 3, 4, 7])
# Draw the circuit
print(qc)
```